Lab Assignment 5

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Mridul Sharma

Roll No. 1024150204

Batch: 2O23

Q1-Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Base {
public:
    void showBase() {
        cout << "This is Base class\n";
    }
};

class Derived : public Base {
public:
    void showDerived() {
        cout << "This is Derived class\n";
    }
};
int main() {
    Base b;
    Derived d;
    b.showBase();
    d.showBase();
    d.showDerived();
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class Base {
public:
    void showBase() {
        cout << "This is Base class\n";
    }
};

class Derived : public Base {
public:
    void showDerived() {
        cout << "This is Derived class\n";
    }
};
int main() {
    Base b;
    Derived d;
    b.showBase();
    d.showBase();
    d.showDerived();
    return 0;
}
```

```
Output
This is Base class
This is Base class
This is Derived class


=== Code Execution Successful ===
```

Q2-Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Base {
protected:
    int num;
public:
    Base() {
        num = 100;
```

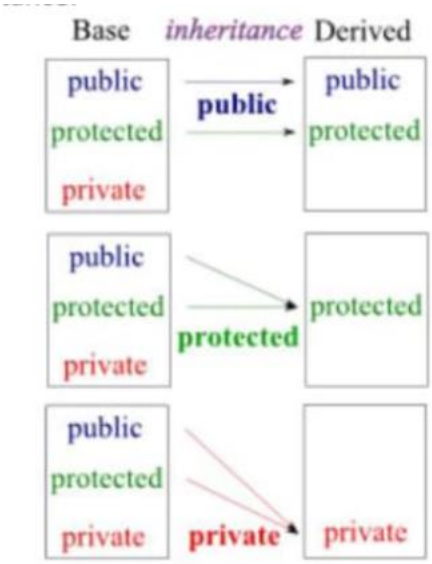
```
    }
};
class Derived : public Base {
public:
    void display() {
        cout << "Protected num = " << num << endl;
    }
};
int main() {
    Derived d;
    d.display();
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class Base {
protected:
    int num;
public:
    Base() {
        num = 100;
    }
};
class Derived : public Base {
public:
    void display() {
        cout << "Protected num = " << num << endl;
    }
};
int main() {
    Derived d;
    d.display();
    return 0;
}
```

```
Output
Protected num = 100


=== Code Execution Successful ===
```

Q3-Most of the time we use public mode of inheritance, for example class Derived : public Base{}. Try protected and private access modifiers to understand the difference of various modes of inheritance.



WRITTEN CODE

```
#include <iostream>

using namespace std;

class Base {

public:

   int a = 10;

};

class PublicD : public Base {};

class ProtectedD : protected Base {};

class PrivateD : private Base {};

int main() {
```

```
    PublicD obj1;
    cout << "Public Inheritance: " << obj1.a << endl;
    // ProtectedD obj2; → cannot access a
    // PrivateD obj3; → cannot access a
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class Base {
public:
    int a = 10;
};
class PublicD : public Base {};
class ProtectedD : protected Base {};
class PrivateD : private Base {};
int main() {
    PublicD obj1;
    cout << "Public Inheritance: " << obj1.a << endl;
    // ProtectedD obj2; → cannot access a
    // PrivateD obj3; → cannot access a
    return 0;
}
```

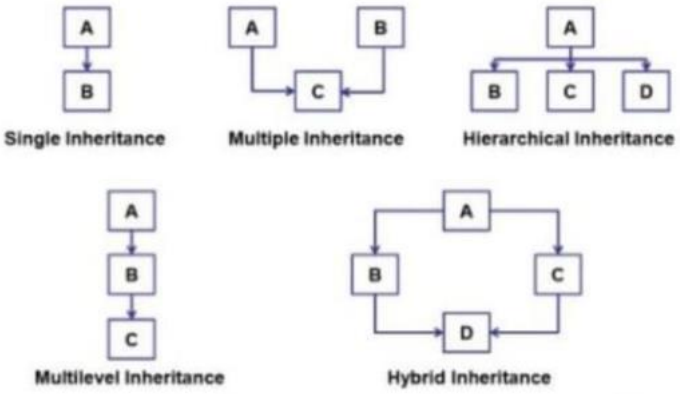
Output

```
Public Inheritance: 10


=== Code Execution Successful ===
```

Q4-Various types of inheritances are as shown below. Write small C++ codes for each inheritance type:

- **Single Inheritance**
- **Multiple Inheritance**
- **Hierarchical Inheritance**
- **Multilevel Inheritance**
- **Hybrid Inheritance**



Single Inheritance:

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "This is class A\n";
    }
};

class B : public A {
public:
    void showB() {
        cout << "This is class B (Derived from A)\n";
    }
};

int main() {
    B obj;
    obj.showA();
    obj.showB();

    return 0;
}
```

```
This is class A
This is class B (Derived from A)


=== Code Execution Successful ===
```

Multiple Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B {
public:
    void showB() {
        cout << "Class B\n";
    }
};
class C : public A, public B {
public:
    void showC() {
        cout << "Class C (Derived from A and B)\n";
    }
};
int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();

    return 0;
}
```

```
Class A
Class B
Class C (Derived from A and B)


=== Code Execution Successful ===
```

Hierarchical Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "This is base class A\n";
    }
};
class B : public A {
public:
    void showB() {
        cout << "This is class B\n";
    }
};
class C : public A {
public:
    void showC() {
        cout << "This is class C\n";
    }
};
int main() {
    B obj1;
    C obj2;
    obj1.showA();
    obj1.showB();
    obj2.showA();
    obj2.showC();
    return 0;
}
```

```
This is base class A
This is class B
This is base class A
This is class C


=== Code Execution Successful ===
```

Multilevel Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B : public A {
public:
    void showB() {
        cout << "Class B (Derived from A)\n";
    }
};
class C : public B {
public:
    void showC() {
        cout << "Class C (Derived from B)\n";
    }
};
int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
    return 0;
}
```

```
Class A
Class B (Derived from A)
Class C (Derived from B)


=== Code Execution Successful ===
```

Hybrid Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void show() {
        cout << "Class A\n";
    }
};
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {
public:
    void display() {
        cout << "Class D (Hybrid Inheritance)\n";
    }
};
int main() {
    D obj;
    obj.show();
    obj.display();
    return 0;
}
```

```
Class A
Class D (Hybrid Inheritance)


=== Code Execution Successful ===
```

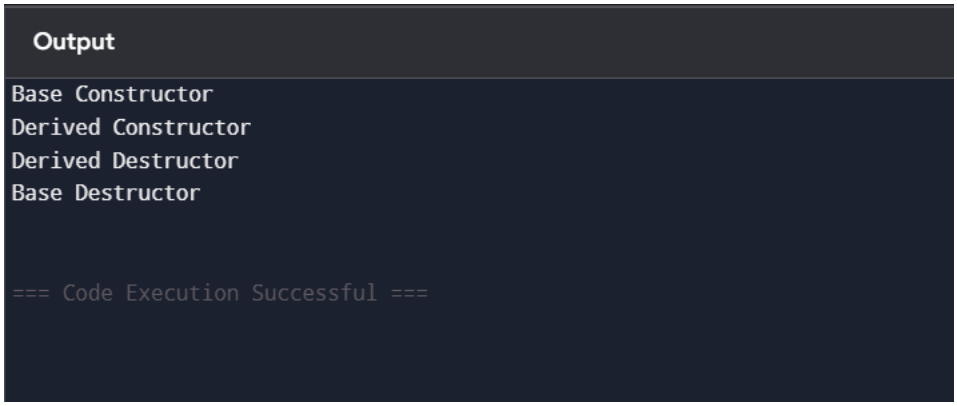
Q5-How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Base {
public:
    Base() {
        cout << "Base Constructor\n";
    }
    ~Base() {
        cout << "Base Destructor\n";
    }
};
class Derived : public Base {
public:
    Derived() {
```

```
        cout << "Derived Constructor\n";
    }
    ~Derived() {
        cout << "Derived Destructor\n";
    }
};
int main() {
    Derived d;
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class Base {
public:
    Base() {
        cout << "Base Constructor\n";
    }
    ~Base() {
        cout << "Base Destructor\n";
    }
};
class Derived : public Base {
public:
    Derived() {
        cout << "Derived Constructor\n";
    }
    ~Derived() {
        cout << "Derived Destructor\n";
    }
};
int main() {
    Derived d;
    return 0;
}
```



Q6-Implement a base class Book with attributes *title, author, and price*. Then, create a derived class Textbook that inherits from Book and adds a new attribute *subject*. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Book {
protected:
    string title, author;
    float price;
public:
    void getBook() {
        cout << "Enter title, author, price:\n";
        cin >> title >> author >> price;
    }
};
class Textbook : public Book {
    string subject;
public:
    void getTextbook() {
        cout << "Enter subject:\n";
        cin >> subject;
```

```
    }
    void display() {
        cout << "\nTitle: " << title
             << "\nAuthor: " << author
             << "\nPrice: " << price
             << "\nSubject: " << subject << endl;
    }
};
int main() {
    Textbook t;
    t.getBook();
    t.getTextbook();
    t.display();
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class Book {
protected:
    string title, author;
    float price;
public:
    void getBook() {
        cout << "Enter title, author, price:\n";
        cin >> title >> author >> price;
    }
};
class Textbook : public Book {
    string subject;
public:
    void getTextbook() {
        cout << "Enter subject:\n";
        cin >> subject;
    }
    void display() {
        cout << "\nTitle: " << title
             << "\nAuthor: " << author
             << "\nPrice: " << price
             << "\nSubject: " << subject << endl;
    }
};
int main() {
    Textbook t;
    t.getBook();
    t.getTextbook();
    t.display();
    return 0;
}
```

Output Clear

```
Enter title, author, price:
DM Harish 300
Enter subject:
Mathematics

Title: DM
Author: Harish
Price: 300
Subject: Mathematics


=== Code Execution Successful ===
```

Q7-A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature.

- **The speed is controlled by a Speedometer class**
- **The fuel level by a FuelGauge class**
- **The temperature by a Thermometer class**

Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard.

Demonstrate how multiple inheritance is used to build this class.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Speedometer {
protected:
   int speed;
public:
   void setSpeed(int s) { speed = s; }
};
class FuelGauge {
protected:
   int fuel;
public:
   void setFuel(int f) { fuel = f; }
};
class Thermometer {
protected:
   int temperature;
public:
   void setTemp(int t) { temperature = t; }
};
```

```
class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    void display() {
        cout << "Speed: " << speed << " km/h\n";
        cout << "Fuel: " << fuel << " %\n";
        cout << "Temperature: " << temperature << " C\n";
    }
};
int main() {
    CarDashboard car;
    car.setSpeed(80);
    car.setFuel(60);
    car.setTemp(35);
    car.display();
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class Speedometer {
protected:
    int speed;
public:
    void setSpeed(int s) { speed = s; }
};
class FuelGauge {
protected:
    int fuel;
public:
    void setFuel(int f) { fuel = f; }
};
class Thermometer {
protected:
    int temperature;
public:
    void setTemp(int t) { temperature = t; }
};
class CarDashboard : public Speedometer, public FuelGauge, public Thermometer
    {
public:
    void display() {
        cout << "Speed: " << speed << " km/h\n";
        cout << "Fuel: " << fuel << " %\n";
        cout << "Temperature: " << temperature << " C\n";
    }
};
int main() {
```

```cpp
int main() {
    CarDashboard car;
    car.setSpeed(80);
    car.setFuel(60);
    car.setTemp(35);
    car.display();
    return 0;
}
```

Output Clear

```
Speed: 80 km/h
Fuel: 60 %
Temperature: 35 C


=== Code Execution Successful ===
```

Q8-You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as *name, ID, and contact details*.

There are two specific types of users:

- **Student**
- **Teacher**

Each type of user has additional attributes:

- **Students → *grade level***
- **Teachers → *department***

Implement a base class LibraryUser with general attributes. Then, create two derived classes Student and Teacher that inherit from LibraryUser and add their own specific attributes.

Demonstrate how hierarchical inheritance is applied in this scenario.

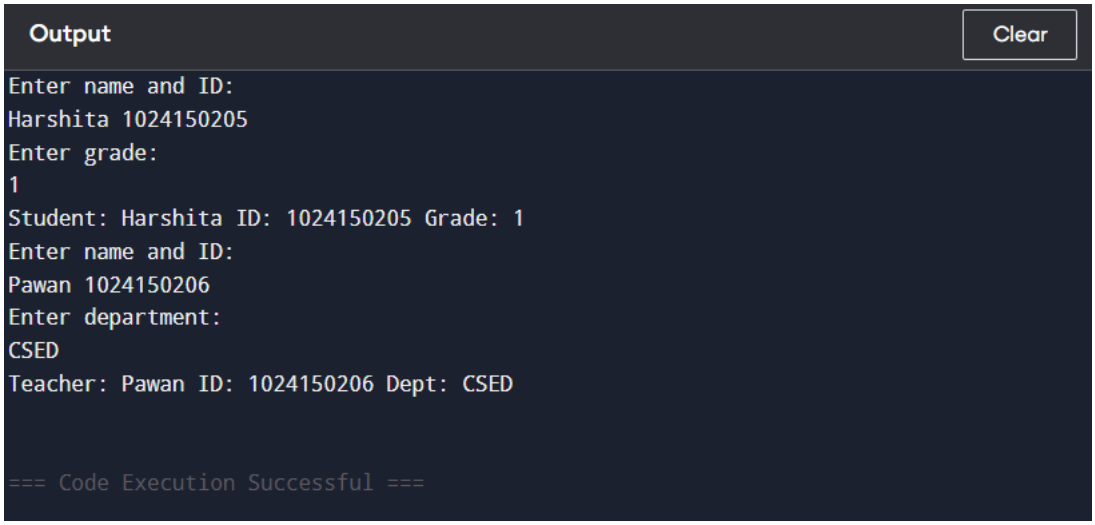
WRITTEN CODE

```
#include <iostream>
using namespace std;
class LibraryUser {
protected:
   string name;
   int id;
public:
   void getUser() {
      cout << "Enter name and ID:\n";
      cin >> name >> id;
   }
};
class Student : public LibraryUser {
   int grade;
public:
   void getStudent() {
      cout << "Enter grade:\n";
      cin >> grade;
```

```
    }
    void display() {
        cout << "Student: " << name << " ID: " << id << " Grade: " << grade << endl;
    }
};
class Teacher : public LibraryUser {
    string dept;
public:
    void getTeacher() {
        cout << "Enter department:\n";
        cin >> dept;
    }
    void display() {
        cout << "Teacher: " << name << " ID: " << id << " Dept: " << dept << endl;
    }
};
int main() {
    Student s;
    Teacher t;
    s.getUser();
    s.getStudent();
    s.display();
    t.getUser();
    t.getTeacher();
    t.display();
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;
class LibraryUser {
protected:
    string name;
    int id;
public:
    void getUser() {
        cout << "Enter name and ID:\n";
        cin >> name >> id;
    }
};
class Student : public LibraryUser {
    int grade;
public:
    void getStudent() {
        cout << "Enter grade:\n";
        cin >> grade;
    }
    void display() {
        cout << "Student: " << name << " ID: " << id << " Grade: " << grade <<
            endl;
    }
};
class Teacher : public LibraryUser {
    string dept;
public:
    void getTeacher() {
        cout << "Enter department:\n";
        cin >> dept;
    }
```

```cpp
    void display() {
        cout << "Teacher: " << name << " ID: " << id << " Dept: " << dept <<
            endl;
    }
};
int main() {
    Student s;
    Teacher t;
    s.getUser();
    s.getStudent();
    s.display();
    t.getUser();
    t.getTeacher();
    t.display();
    return 0;
}
```



Q9-A logistics company needs a software system to manage its vehicle fleet.

- **All vehicles share common attributes like *make, model, and year***
- **Trucks have additional attribute: *load_capacity***
- **Refrigerated trucks have an extra attribute: *temperature_control***

Implement:

- **Base class Vehicle**
- **Derived class Truck**
- **Derived class RefrigeratedTruck (inherits from Truck)**

Demonstrate how multilevel inheritance works in this case.

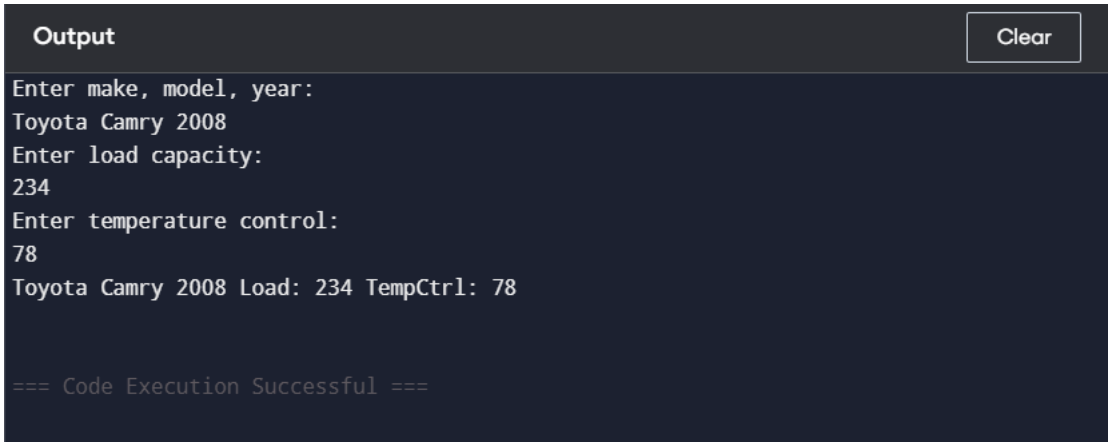
WRITTEN CODE

```
#include <iostream>
using namespace std;
class Vehicle {
protected:
    string make, model;
    int year;
public:
    void getVehicle() {
        cout << "Enter make, model, year:\n";
        cin >> make >> model >> year;
```

```
    }
};
class Truck : public Vehicle {
protected:
    int load_capacity;
public:
    void getTruck() {
        cout << "Enter load capacity:\n";
        cin >> load_capacity;
    }
};
class RefrigeratedTruck : public Truck {
    int temperature_control;
public:
    void getRefrigerated() {
        cout << "Enter temperature control:\n";
        cin >> temperature_control;
    }
    void display() {
        cout << make << " " << model << " " << year
             << " Load: " << load_capacity
             << " TempCtrl: " << temperature_control << endl;
    }
};
int main() {
    RefrigeratedTruck r;
    r.getVehicle();
    r.getTruck();
    r.getRefrigerated();
```

```
    r.display();
    return 0;
}
```

```
#include <iostream>
using namespace std;
class Vehicle {
protected:
    string make, model;
    int year;
public:
    void getVehicle() {
        cout << "Enter make, model, year:\n";
        cin >> make >> model >> year;
    }
};
class Truck : public Vehicle {
protected:
    int load_capacity;
public:
    void getTruck() {
        cout << "Enter load capacity:\n";
        cin >> load_capacity;
    }
};
class RefrigeratedTruck : public Truck {
    int temperature_control;
public:
    void getRefrigerated() {
        cout << "Enter temperature control:\n";
        cin >> temperature_control;
    }
    void display() {
        cout << make << " " << model << " " << year
             << " Load: " << load_capacity
             << " TempCtrl: " << temperature_control << endl;
    }
};
int main() {
    RefrigeratedTruck r;
    r.getVehicle();
    r.getTruck();
    r.getRefrigerated();
    r.display();
    return 0;
}
```



Q10-You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student.

- **A Person has general attributes like *name and address***
- **Staff (inherits Person) → attributes: *employee_id, department***
- **Student (inherits Person) → attributes: *student_id, grade***

Some Staff members are also Students (e.g., teaching assistants), so they need to inherit from both classes.

Implement:

- **Base class Person**
- **Derived classes Staff and Student**
- **A class TeachingAssistant that inherits from both Staff and Student**

Demonstrate how hybrid inheritance is applied and managed in this scenario.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Person {
protected:
    string name;
public:
    void getPerson() {
        cout << "Enter name:\n";
```

```
        cin >> name;
    }
};
class Staff : virtual public Person {
protected:
    int emp_id;
public:
    void getStaff() {
        cout << "Enter employee ID:\n";
        cin >> emp_id;
    }
};
class Student : virtual public Person {
protected:
    int student_id;
public:
    void getStudent() {
        cout << "Enter student ID:\n";
        cin >> student_id;
    }
};
class TeachingAssistant : public Staff, public Student {
public:
    void display() {
        cout << "Name: " << name
             << " EmpID: " << emp_id
             << " StudentID: " << student_id << endl;
    }
};
```

```
int main() {
    TeachingAssistant ta;
    ta.getPerson();
    ta.getStaff();
    ta.getStudent();
    ta.display();
    return 0;
}
```

```
#include <iostream>
using namespace std;
class Person {
protected:
    string name;
public:
    void getPerson() {
        cout << "Enter name:\n";
        cin >> name;
    }
};
class Staff : virtual public Person {
protected:
    int emp_id;
public:
    void getStaff() {
        cout << "Enter employee ID:\n";
        cin >> emp_id;
    }
};
class Student : virtual public Person {
protected:
    int student_id;
public:
    void getStudent() {
        cout << "Enter student ID:\n";
        cin >> student_id;
    }
};
class TeachingAssistant : public Staff, public Student {
public:
```

```
    void display() {
        cout << "Name: " << name
             << " EmpID: " << emp_id
             << " StudentID: " << student_id << endl;
    }
};
int main() {
    TeachingAssistant ta;
    ta.getPerson();
    ta.getStaff();
    ta.getStudent();
    ta.display();
    return 0;
}
```

Output Clear

```
Enter name:
Harshita
Enter employee ID:
1024150205
Enter student ID:
10241
Name: Harshita EmpID: 1024150205 StudentID: 10241


=== Code Execution Successful ===
```